Animación del algoritmo Heap Sort en Python

Ma. Alejandra Ramirez Torres
Ingeniería de Software
Universidad de La Salle
Bogotá, Colombia
mramirez45@unisalle.edu.co

Juan Fernando Carreño
Ingeniería de Software
Universidad de La Salle
Bogotá, Colombia
jcarreno86@unisalle.edu.co

Daniel Cubillos
Ingeniería de Software
Universidad de La Salle
Bogotá, Colombia
dcubillos81@unisalle.edu.co

***Resumen*—Este documento presenta el desarrollo de una animación del algoritmo de ordenamiento Heap Sort hecha en Python con la librería Pygame. Cada barra representa un personaje de películas de terror, con un color característico, y su altura es un nivel de poder ficticio de 1 a 15. Un brazo mecánico agarra las dos barras que se están inspeccionando para que se entienda mejor qué hace el algoritmo, y en pantalla se ven las comparaciones, los intercambios y las posiciones que ya quedaron ordenadas. Para separar la lógica del algoritmo de la parte gráfica, Heap Sort se implementó como un generador, de modo que cada paso se dibuja en un cuadro distinto. Se midió el número de comparaciones e intercambios con varios órdenes iniciales y se comprobó un crecimiento cercano a n log n. El resultado es una herramienta para explicar el algoritmo en una exposición de clase.**

Palabras clave—Heap Sort, algoritmos de ordenamiento, animación, Pygame, Python, visualización

I. Introducción

Los algoritmos de ordenamiento son de los primeros temas que se ven en ingeniería de software, y aun así a veces cuesta imaginar qué pasa dentro del arreglo mientras el algoritmo trabaja. Una animación ayuda bastante, porque permite ver cada comparación y cada intercambio mientras ocurren [6].

En este trabajo se desarrolló una animación de Heap Sort, algoritmo propuesto por Williams en 1964 [1], usando Python y la librería Pygame [4]. Como el programa se iba a exponer, se buscó una visualización minimalista pero con buena presentación, y se ambientó en Halloween: cada barra es un personaje de películas de terror con un color propio. En las siguientes secciones se explican el algoritmo, la implementación y las pruebas que se hicieron.

II. Marco teórico

A. Montículo binario

Un montículo binario máximo (*max-heap*) es un árbol binario casi completo que se guarda en un arreglo. Para el elemento en la posición i, los hijos están en $2i+1$ y $2i+2$, y cada padre es mayor o igual que sus hijos, así que el mayor elemento siempre queda en la posición 0 [2].

B. Algoritmo Heap Sort

Heap Sort tiene dos fases. Primero se construye el max-heap aplicando la operación *heapify* desde el último nodo que tiene hijos hasta la raíz. Después se intercambia la raíz (el máximo) con el último elemento del montículo, se reduce el tamaño del montículo en uno y se aplica *heapify* en la raíz. Esto se repite hasta que el montículo queda vacío y el arreglo queda ordenado de menor a mayor.

C. Complejidad

Construir el montículo cuesta O(n) y cada una de las n extracciones cuesta O(log n), por lo que el algoritmo es O(n log n) en el peor caso, en el caso promedio y en el mejor. Ordena sobre el mismo arreglo (memoria extra O(1)), pero no es estable [2], [3].

III. Implementación

A. Herramientas

El programa está escrito en Python 3 y usa Pygame para la ventana, el dibujo y el teclado, sin otras librerías. El código se dividió en tres partes: el algoritmo, las funciones de dibujo y el ciclo principal.

B. Heap Sort como generador

Si el algoritmo se ejecutara de una sola vez, en pantalla solo se vería el resultado final. Por eso *heapify* y *heap_sort* se escribieron como generadores de Python [5]: cada vez que el algoritmo compara, intercambia o marca una posición como ordenada, entrega un evento con *yield*. El ciclo principal de Pygame pide un evento cada cierto tiempo, según la velocidad elegida, y dibuja el estado del arreglo. Así la lógica queda separada de lo gráfico y se puede pausar o acelerar sin cambiar el algoritmo.

C. Diseño visual

La ventana es de 1100×680 píxeles, con fondo degradado de morado a café oscuro, una luna, telarañas en las esquinas, murciélagos en movimiento y una calabaza junto al título. Cada barra es un personaje y su color se escogió a partir de algo característico de él (Tabla I). A cada personaje se le asignó un valor de 1 a 15 según su nivel de poder, que es ficticio y lo definimos nosotros (1 el más débil, 15 el más fuerte). Ese valor es el que se ordena: la altura de la barra es proporcional a él, el número se ve dentro de la barra y el nombre aparece debajo en vertical.

Durante la animación las barras que se comparan se aclaran y brillan en amarillo, las que se intercambian se aclaran más y las que ya están en su posición final se marcan con un borde verde. En la parte de abajo se muestran la fase actual, las comparaciones, los intercambios y la velocidad.

TABLA I. Personajes, poder y colores de las barras

Poder	Personaje	Color	Referencia

1	Zombie	Verde oliva	Piel en descomposición
2	Ghostface	Blanco	Máscara blanca
3	Jack el Destripador	Gris niebla	Niebla de Londres
4	Jigsaw	Rojo	Muñeco Billy
5	Chucky	Naranja	Cabello pelirrojo
6	Annabelle	Rosa pálido	Vestido de la muñeca
7	Frankenstein	Verde	Piel verde
8	Michael Myers	Azul	Overol azul
9	Jason	Gris beige	Máscara de hockey
10	Drácula	Vino	Capa
11	Freddy Krueger	Rojo oscuro	Suéter rojo y verde
12	La Monja	Gris oscuro	Hábito
13	Slenderman	Gris	Traje gris
14	Valak	Morado	Aura demoníaca
15	Pennywise	Rojo	Globo rojo

D. Brazo mecánico

Para que se entienda mejor qué está haciendo el algoritmo, se imaginó un brazo mecánico que agarra las dos barras que se están inspeccionando en ese momento. Está formado por un riel en la parte de arriba, un carro que se desplaza por el riel, una viga y dos brazos telescópicos con una pinza cada uno. Cuando el algoritmo compara dos posiciones, las pinzas bajan y se apoyan sobre esas dos barras. Cuando las intercambia, las pinzas se cierran, levantan las barras y las llevan hasta la posición del otro, y luego las sueltan. Si no hay nada que inspeccionar (al inicio o al terminar), el brazo sube y se queda en reposo.

El movimiento de las barras y del brazo se suaviza en cada cuadro acercando la posición dibujada a la posición real, de modo que el algoritmo sigue trabajando por pasos y la animación solo se encarga de mostrarlo.

E. Controles

La barra espaciadora inicia o pausa la animación; R genera un orden aleatorio, N uno casi ordenado y V uno inverso; las teclas + y − cambian la velocidad.

IV. Resultados

La Fig. 1 muestra el brazo mecánico en pleno intercambio, con las pinzas cerradas y las dos barras levantadas, y la Fig. 2 muestra el resultado final con todas las barras ordenadas por poder y el brazo en reposo. En todas las pruebas el arreglo quedó bien ordenado (se comparó contra la función *sorted* de Python).

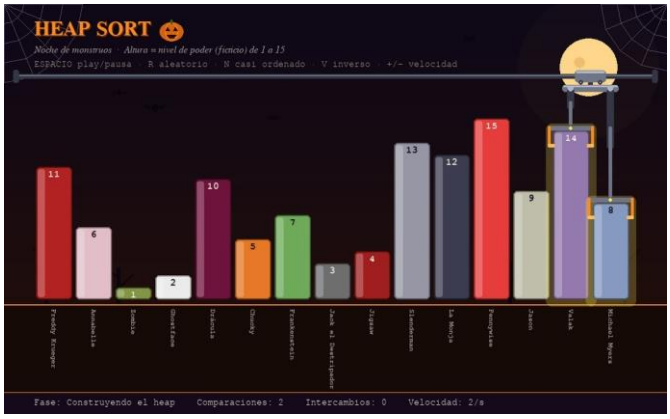


Fig. 1. El brazo agarra y levanta las dos barras que se intercambian.

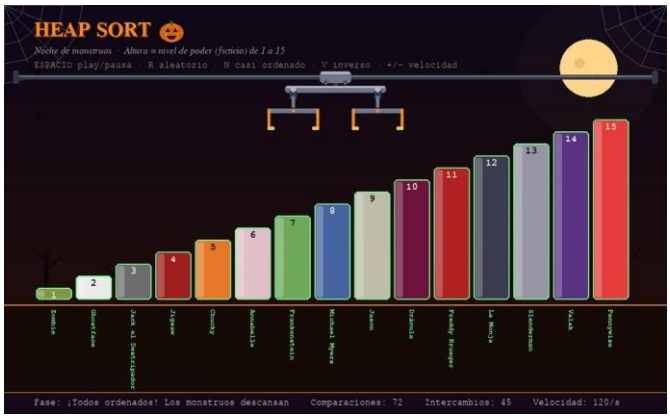


Fig. 2. Resultado final: barras ordenadas de menor a mayor poder.

Para ver cómo se comporta con distintos órdenes iniciales, se ejecutó el algoritmo 1000 veces con los 15 personajes y se contaron las comparaciones y los intercambios (Tabla II). El orden inverso siempre dio 66 comparaciones, porque un arreglo descendente ya es un max-heap y la primera fase no hace intercambios. Los otros casos quedaron cerca entre sí, lo que concuerda con que Heap Sort casi no depende del orden inicial.

TABLA II. Operaciones con 15 elementos (1000 ejecuciones)

Orden inicial	Comp. (prom.)	Rango	Interc. (prom.)
Aleatorio	72.4	62–78	46.7
Casi ordenado	75.0	67–80	50.1
Inverso	66.0	66–66	39.0
Ordenado	77.0	77–77	52.0

También se probó con arreglos aleatorios más grandes (Tabla III). Las comparaciones se mantuvieron entre 1.2 y 1.8 veces $n \log_2 n$, es decir que crecen como $n \log n$, como dice la teoría.

TABLA III. Comparaciones según el tamaño del arreglo

n	Comparaciones	$n \log_2 n$	Razón
15	73	59	1.24
100	1032	664	1.55
1000	16842	9966	1.69
5000	107697	61439	1.75

V. Conclusiones

Se logró una animación de Heap Sort que muestra cómo se construye el montículo y cómo se van sacando los máximos. Escribir el algoritmo como generador fue lo que más ayudó, porque separó la lógica del dibujo y facilitó pausar y cambiar la velocidad. El brazo mecánico ayuda a ver de un vistazo cuáles son las dos barras que se están comparando o intercambiando, y la ambientación de Halloween, con personajes y colores propios, hace la animación más fácil de seguir en la exposición, aunque algunos colores se parecen (por ejemplo Frankenstein y Zombie, ambos verdes) y se pueden confundir de lejos (por eso el valor también está escrito dentro de cada barra).

Como trabajo futuro se podrían agregar otros algoritmos, como Quick Sort y Merge Sort, para compararlos lado a lado, y una opción para elegir el número de barras.

Referencias

[1] J. W. J. Williams, “Algorithm 232: Heapsort,” Commun. ACM, vol. 7, no. 6, pp. 347–348, 1964.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA: MIT Press, 2009.

[3] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA: AddisonWesley, 2011.

[4] Pygame Community, “Pygame documentation.” [Online]. Available: https://www.pygame.org/docs/

[5] Python Software Foundation, “The Python Tutorial: Generators.” [Online]. Available: https://docs.python.org/3/tutorial/classes.html#generators

[6] Toptal, “Heap sort,” Sorting Algorithms Animations. [Online]. Available: https://www.toptal.com/developers/sorting-algorithms/heap-sort